

DSA STUDY BLUEPRINT SERIES

88. Maximum Width of Binary Tree

Calculating Maximum Width of Binary Trees

Section 09 • C++ Core Data Structures & Algorithms

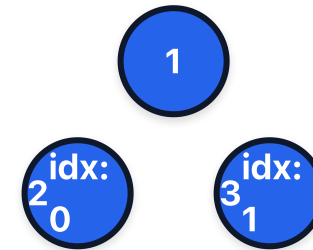
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Width:** Max nodes count at any level, including null nodes.
- Assign indices: left = $2 * \text{idx}$, right = $2 * \text{idx} + 1$.

Memory & Execution Blueprint



$$\text{Width} = 1 - 0 + 1 = 2$$

C++ Implementation Reference

Standard code layout and syntax

```
// BFS using queue> (node, index).  
// Max width = max(width, right_idx - left_idx + 1) at each level.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(W)$ (Maximum width nodes)

Key Practices & Gotchas

- **Important:** Subtract base index at each level to prevent integer overflow in deep trees.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.